

Networking

Introduction

Symbian Technical Day, Warsaw
16th of June, 2004



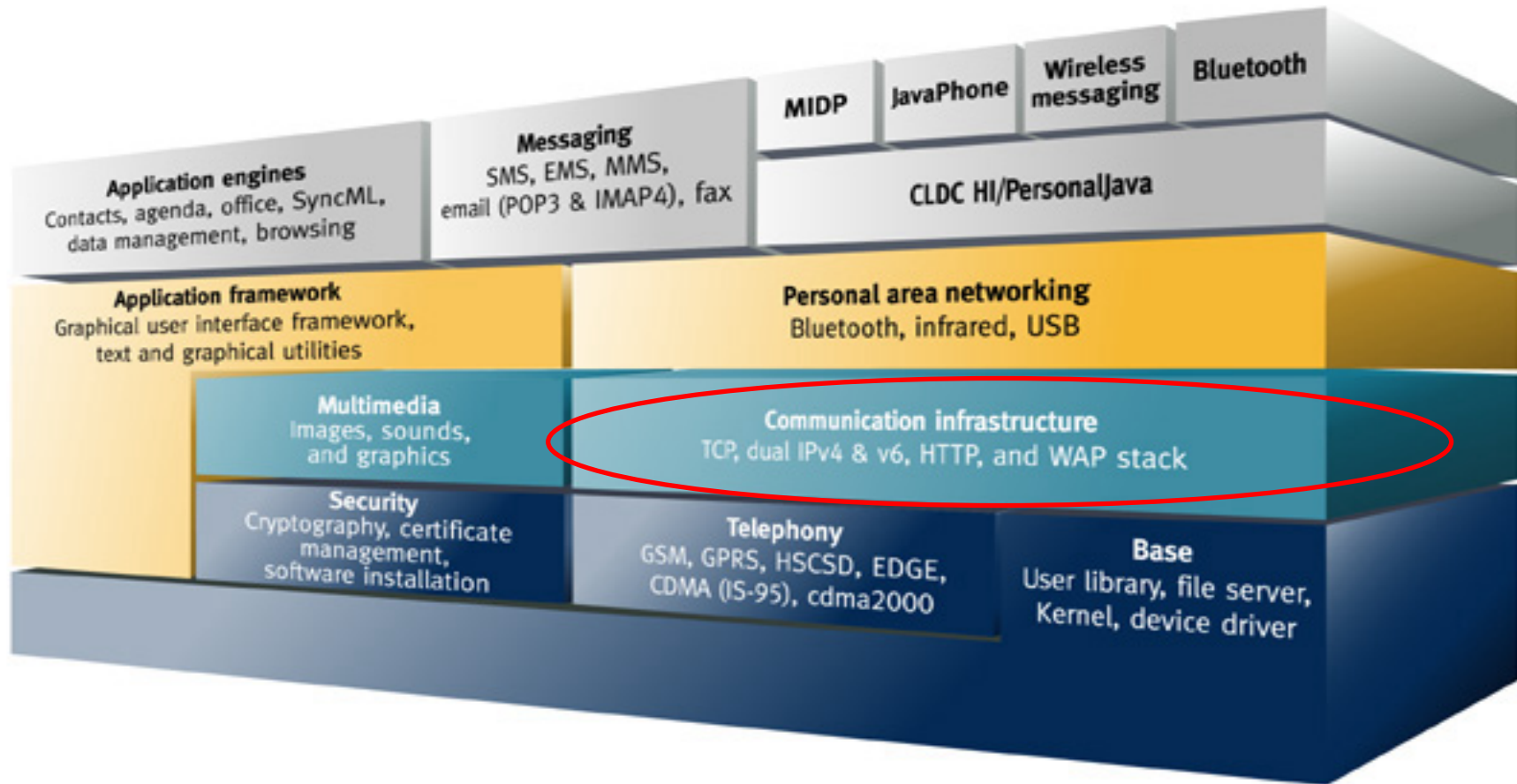
Marko Okkonen, Senior Technology Expert, Forum Nokia

NOKIA

Outline

- **Introduction to Networking**
- Connection Management
- Socket Communication
- HTTP Client API
- Communications Database
- More Information

Introduction to Networking

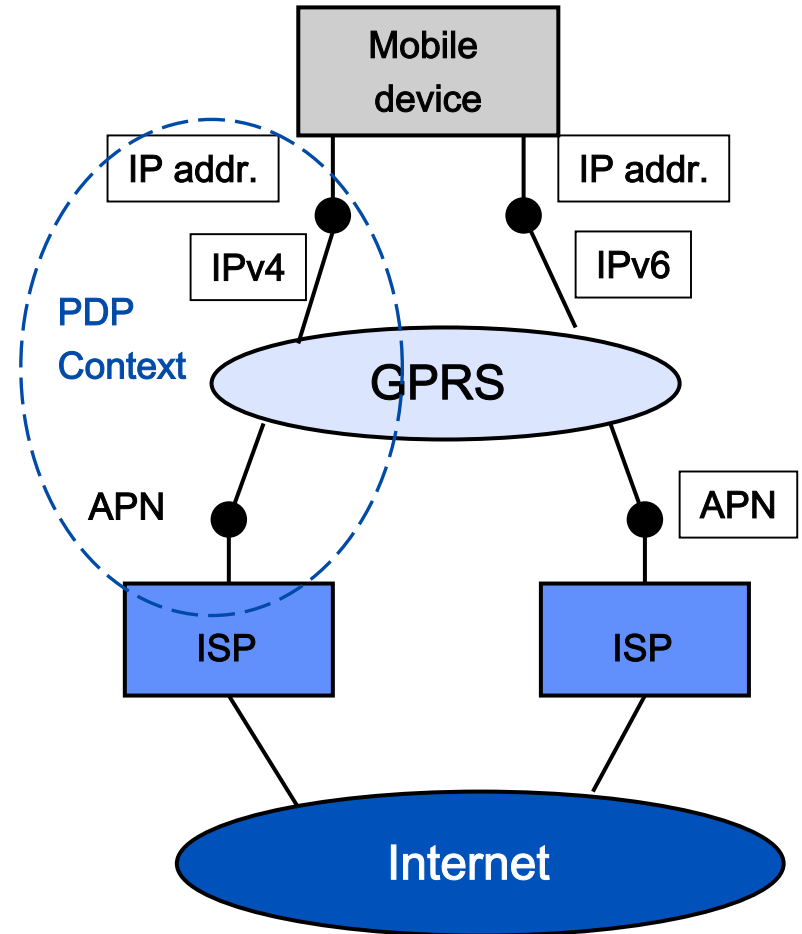


Networking Overview

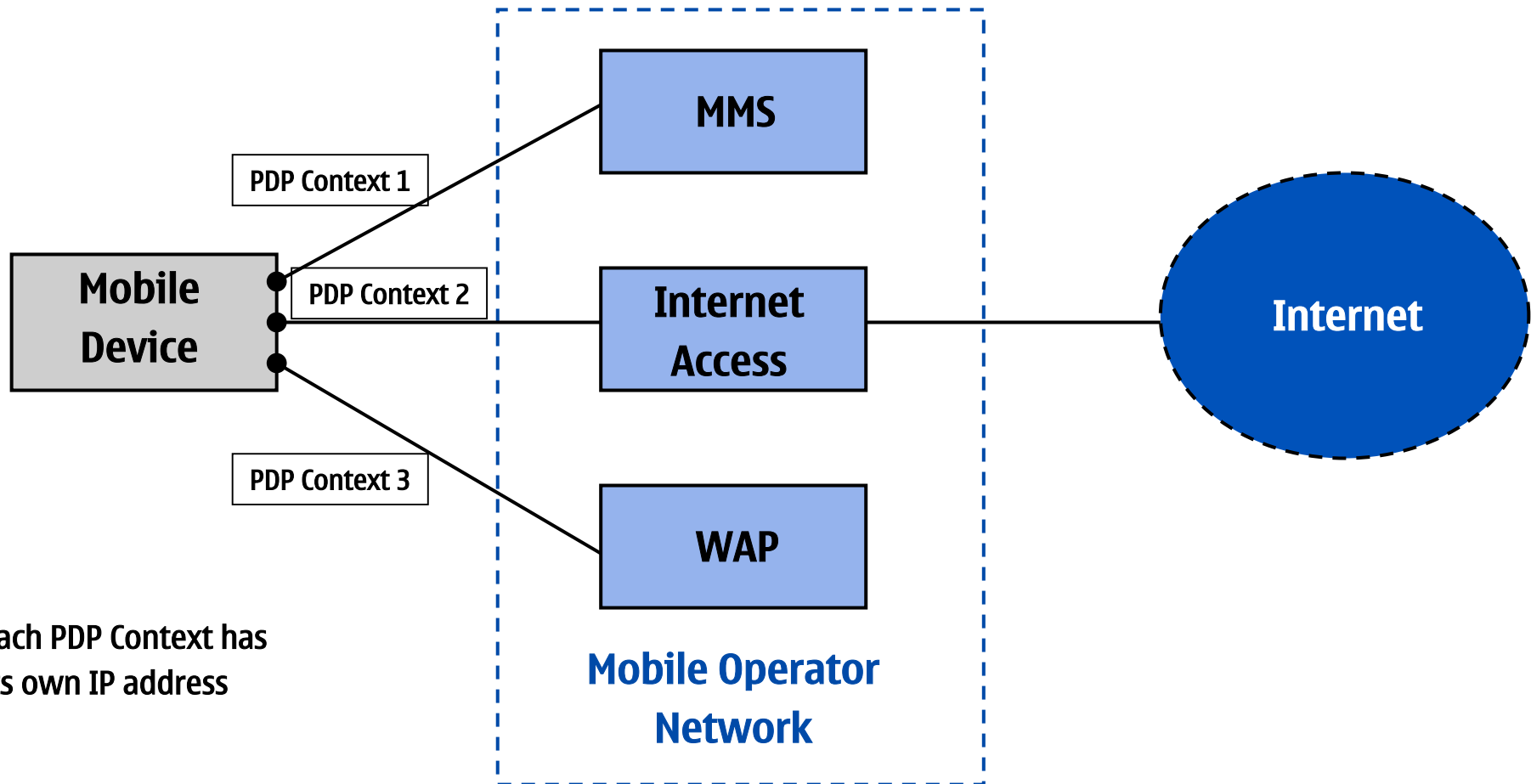
- Features
 - IPv4/IPv6 dual stack
 - HTTP/1.1 client stack
 - Multihoming (i.e. multiple PDP contexts)
 - Connection Management API
 - Connection Manager application
- Bearers supported
 - GSM Circuit Switched Data
 - GSM High-Speed Circuit Switched Data
 - GPRS
 - EDGE
 - WLAN (Series 80)
 - WCDMA (Series 60 2nd Edition Feature Pack 2)

Mobile Internet over GPRS

- “Always on”
- PDP context
 - PDN type (e.g. IPv4)
 - PDN address (e.g. IP address)
 - QoS
 - APN
- Single-/multihoming
- Usually dynamic IP address, because of Network Address Translation Gateway (NAT), which implementation varies between the operators



Multihoming



WCDMA

- Series 60 2nd Edition Feature Pack 2 introduces WCDMA support
- Max bitrates: 384 kbps downlink, 128 kbps uplink
- Allows video telephone, faster browsing, better quality on audio/video streaming
- Enables simultaneous voice call and use of WCDMA network
- 3GPP Rel'99 WCDMA/GPRS Dual mode

Outline

- Introduction to Networking
- **Connection Management**
- Socket Communication
- HTTP Client API
- CommDB
- More Information

Connection Management Overview

- Connection Management API is implemented by `RConnection` class
- The API provides an interface for creating and managing connections
- Supports multi-homing, which allows multiple connections to be active simultaneously
- Connection Manager built-in application provided by the platform

RConnection

- Opening and closing the connection
- Starting a connection, which means associating it with a new underlying interface
- Attaching the **RConnection** instance to an existing interface
- Obtaining progress information and notification during connection start-up
- Notifying when subconnections come up and go down
- Reading CommDB fields specific to an active connection
- Collecting statistical information on the network connection and subconnections

Implicit Connection Startup

```
RSocketServ ss;  
RSocket sock;  
TRequestStatus status;  
  
// Connect to the socket server  
ss.Connect();  
  
// Open a TCP socket  
sock.Open(ss, KAfInet, KSockStream, KProtocolInetTcp);  
  
_LIT(KRasAddr, "10.159.24.13");  
const TInt KEchoPort = 7;  
  
TInetAddr destAddr;  
destAddr.Input(KRasAddr);  
destAddr.SetPort(KEchoPort);  
  
// Request the Socket to connect to the destination (implicit Connection)  
sock.Connect(destAddr, status);
```

Explicit Connection Setup

```
RSocketServ ss;
RConnection conn;
TRequestStatus status;
_LIT(KRasAddr, "10.159.24.13");
const TInt KEchoPort = 7;

TInetAddr destAddr;
destAddr.Input(KRasAddr);
destAddr.SetPort(KEchoPort);

// Connect to the socket server
ss.Connect();

// Open an RConnection object.

conn.Open(ss);

// Create overrides
TCommDbConnPref prefs
prefs.SetDialogPreference(ECommDbDialogPrefDoNotPrompt);
prefs.SetDirection(ECommDbConnectionDirectionOutgoing);
prefs.SetIapId(4);

// Start an Outgoing Connection with overrides
conn.Start(prefs);

// Open a Socket associated with the connection
RSocket sock;
sock.Open(ss, KAfInet, KSockStream, KProtocolInetTcp, conn);

// Request the Socket to connect to the
// destination
sock.Connect(destAddr, status);
```

Attaching to a Connection

```
// Open a session with the socket server
RSocketServ socketServer;
TInt err(KErrNone);
err = socketServer.Connect();
If (err != KErrNone) return err;

//Open a subsession with the socket server for the Rconnection
RConnection myConnection;
err = myConnection.Open(socketServer);
If (err != KErrNone) return err;

//Start or attach the RConnection to an interface on the server
TUint connectionCount;
//Enumerate currently active connections across all socket servers
err = myConnection.EnumerateConnections(connectionCount);
If (err != KErrNone) return err;

If (connectionCount == 0) // if no connections are started
{
    err = myConnection.Start();
}
else
{
    TPckgBuf<TConnectionInfoV2> connectionInfo;
    err = myConnection.GetConnectionInfo(1, connectionInfo); // 1 = first active connection
    if (err!=KErrNone) return err;
    err = myConnection.Attach(connectionInfo, EAttachTypeMonitor);
}
```

Disabling Inactivity Timer

- Connection inactivity timer can be disabled using `RConnection::SetOpt()` function instead of `RNif::DisableTimers()`, which was used before.

```
SetOpt(KCOLProvider, KConnDisableTimers, ETrue);
```

- By default, `LastSocketActivityTimeout` value in Comms Db is set to infinity (-1), so connection will not be closed automatically

Application Behaviour

- User confirmation is required always before connection is made, at least first time
- Application may store its own settings for the default AP if selection dialog is not wanted to be shown. The dialog is only started at first time
- AVOID HARD CODINGS WHERE POSSIBLE!

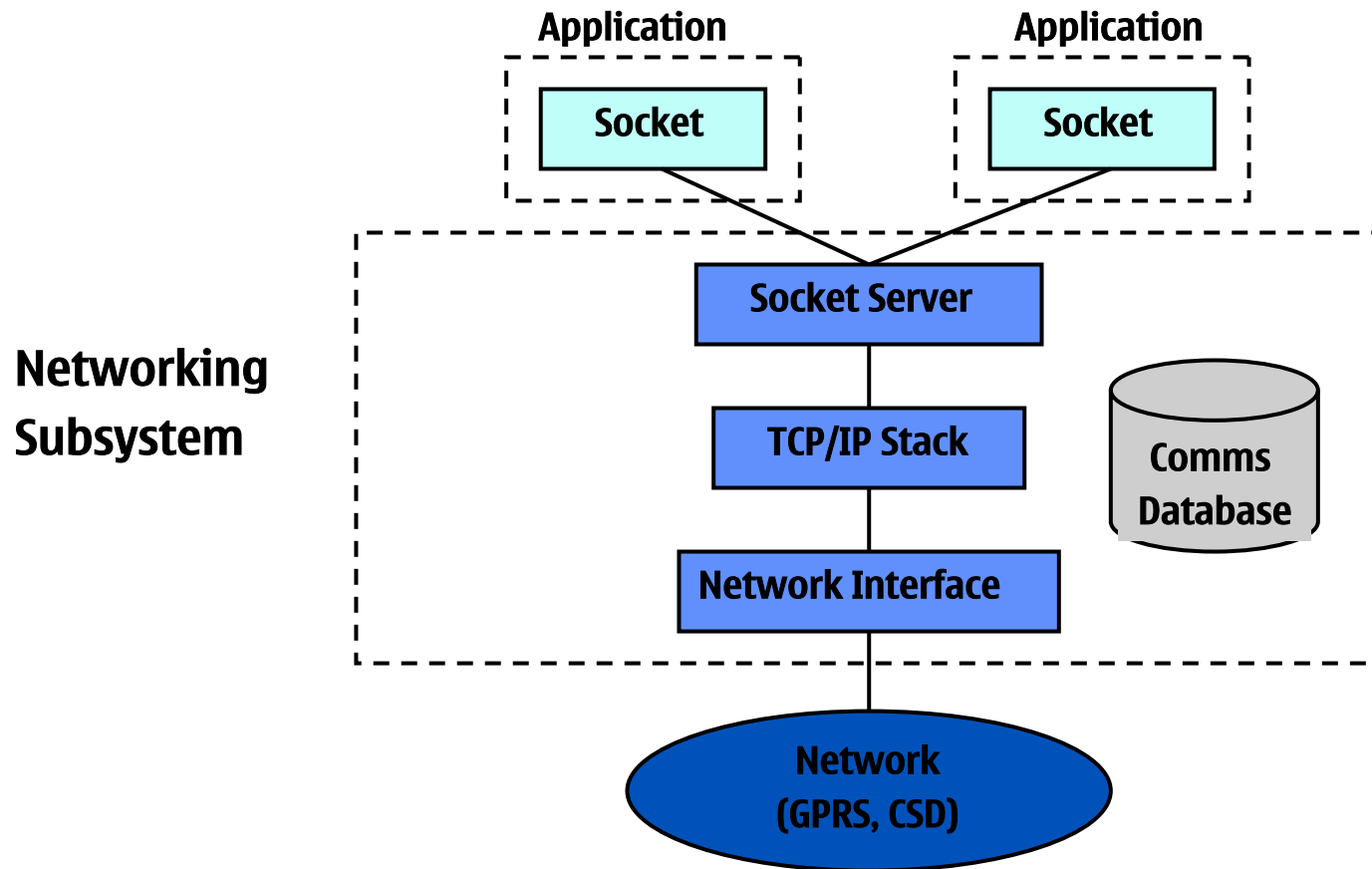
Outline

- Introduction to Networking
- Connection Management
- **Socket Communication**
- HTTP Client API
- Communications Database
- More Information

Socket Communication

- Symbian OS sockets are similar in concept of BSD socket
- The socket architecture provides generic client interface and server, where protocol modules can be plugged in.
- Socket can be used over TCP/IP, IrDa and Bluetooth.
- The socket client API is provided by RSocket and server API by RSocketServ.

Socket Communication



RSocket

- Provides the following services:
 - reading from and writing to protocol
 - binding to addresses
 - active connecting
 - passive connection through the listen/accept model
- Usually used asynchronously using active objects

Using Sockets

```
RSocketServ ss;  
RSocket sock;  
TRequestStatus status;  
  
// Connect to the socket server  
ss.Connect();  
  
// Open a TCP socket  
sock.Open(ss, KAfInet, KSockStream, KProtocolInetTcp);  
  
_LIT(KRasAddr, "10.159.24.13");  
const TInt KEchoPort = 7;  
  
TInetAddr destAddr;  
destAddr.Input(KRasAddr);  
destAddr.SetPort(KEchoPort);  
  
// Request the Socket to connect to the destination (implicit Connection)  
sock.Connect(destAddr, status);  
User::WaitForRequest(status);  
  
// Write to the socket  
sock.Send(data, flags, status);  
User::WaitForRequest(status);  
  
// Close the socket  
sock.Close();
```

Secure Sockets

- Currently supports TLS v1.0 and SSL v3.0 protocol

- **Via SetOpt**

```
RSocket socket;  
socket.Open (...);  
Socket.Connect ();  
socket.SetOpt (KSoSecureSocket, KSolInetSSL);
```

- **Via CSecureSocket**

```
Rsocket socket;  
CSecureSocket tlsSocket;  
socket.Open (...);  
socket.Connect ();  
_LIT (KTLS1, "TLS1.0");  
tlsSocket = CSecureSocket::NewL (socket, KTLS1);  
// start the handshake  
tlsSocket->StartClientHandshake (...);
```

Outline

- Introduction to Networking
- Connection Management
- Socket Communication
- **HTTP Client API**
- Communications Database
- More Information

HTTP Features

- HTTP 1.0 and 1.1 Client stack with support for persistent connections, pipelining, and chunked transfer encoding
- Filters provided to support HTTP Redirection and HTTP Basic and Digest Authentication.
- Secure HTTP Connection supported with SSL 3.0 and TLS 1.0
- HTTP state management (i.e. cookies) is supported

HTTP Client API

- There are five key concepts used in the API:
 - Sessions
 - Transactions
 - Headers
 - Data suppliers
 - Filters

Sessions

- A session encapsulates the client's HTTP activity over the duration of the client's execution.
- Usually one session at time, but can be several
- Each session has a set of properties, which define HTTP protocol, encoding and transport that applies for transactions carried out within the life of the session
- The session has also an associated set of filters
- Session class is provided by RHTTPSession

```
// Open a session using the default protocol HTTP/TCP  
void RHTTPSession::OpenL()
```

Transactions

- A transaction represents an interaction between the HTTP client and server - a client request and a server response
- Transactions are executed asynchronously within the client's process. The client is notified when events are available for each outstanding transaction
- Both the request and response portion of a transaction are composed of a header and an optional body
- The request portion of the transaction also specifies an HTTP Method together with a URI that specifies the resource held at the server
- The response portion of a transaction contains an HTTP status code and message

Transactions - continued

- Transaction class is provided by RHTTPTransaction and callback interface by MHTTPTransactionCallback

```
// Open a transaction
```

```
void RHTTPSession::OpenTransactionL(const TUriC8 &aURI,  
    MHTTPTransactionCallback &aCallback,  
    RStringF aMethod=RStringF())
```

```
// Callback function
```

```
virtual Tint RHTTPTransactionCallback::MHFRunL(  
    RHTTPTransaction aTransaction,  
    const THTTPEvent& aEvent) = 0
```

Transactions – continued 2

```
// Get the response
RHTTPResponse RHTTPTransaction::Response() const

// Get the request
RHTTPResquest RHTTPTransaction::Request() const

// Get the property set
RHTTPTransactionPropertySet
    RHTTPTransaction::PropertySet() const

// Submit the transaction
void RHTTPTransaction::SubmitL(
    THTTPFilterHandle aStart=THTTPFilter::EClient)
```

Headers

- The header may have zero or more fields
- Headers are encoded according the HTTP protocol and encoding that are used
- The headers class is provided by RHTTPHeaders

```
// Get request headers
RHTTPHeaders RHTTPRequest::GetHeaderCollection()
// Set a named field
RHTTPHeaders::SetFieldL(RStringF aFieldName,
                        THTTPHdrVal& aHeaderValue)

// Get response headers
RHTTPHeaders RHTTPResponse::GetHeaderCollection()
// Obtain a named header field's value
TInt RHTTPHeaders::GetField(RStringF aFieldName,
                            TInt aPartIdx, THTTPHdrVal& aHeaderValue) const
```

Data Suppliers

- The body portion of requests and responses is represented in the API as mix-in interface, which allows implementation of the classes that generate body data to be hidden
- API enables signalling between the client and transport in use, to that body data is consumed or emitted at a rate the client can support
- Clients can assemble the body piece by piece
- Data supplier classes must implement MHTTPDataSupplier

Data Suppliers - Continued

```
// Inform HTTP that new data is available
RHTTPTransaction::NotifyNewRequestBodyPartL()

// Obtain a data part from the supplier
TBool MHTTPDataSupplier::GetNextDataPart(TPtrC8& aDataPart) = 0

// Data will survive until ReleaseData() is called
void MHTTPDataSupplier::ReleaseData() = 0
```

Filters

- Filters are add-on modules that provide additional behaviours to a session
- Behaviours may be triggered by
 - The presence of particular headers in a request or a response
 - By status codes in a response
 - Particular events, which occur on a transaction
E.g. client authentication, redirection or caching
- Filter classes must implement `MHTTFilter`

Using RConnection with RHTTPSession

```
User::LeaveIfError(iSocketServ.Connect());
User::LeaveIfError(iConnection.Open(iSocketServ));
iHttpSession.OpenL();

// Initiate connection
iConnection.Start(iStatus);

//...

RHTTPConnectionInfo connInfo = iHttpSession.ConnectionInfo();
RStringPool pool = iHttpSession.StringPool();

// Attach to socket server
connInfo.SetPropertyL(pool.StringF(HTTP::EHttpSocketServ, RHTTPSession::GetTable()),
THTTPHdrVal(iSocketServ.Handle()));

// Attach to connection
TInt connPtr = REINTERPRET_CAST(TInt, &iConnection);
connInfo.SetPropertyL(pool.StringF(HTTP::EHttpSocketConnection,
RHTTPSession::GetTable()), THTTPHdrVal(connPtr));

// Open transaction...
iHttpTrans = iHttpSession.OpenTransactionL(iUri, *this, pool.StringF(HTTP::EGET,
RHTTPSession::GetTable()));
RHTTPHeaders hdr = iHttpTrans.Request().GetHeaderCollection();

//...
```

Outline

- Introduction to Networking
- Connection Management
- Socket Communication
- HTTP Client API
- **Communications Database**
- More Information

Communications Database Overview

- Comms Db stores information about ISPs, modem and GPRS settings, WAP, proxies, Bluetooth, connection preferences etc.
- Comms Db is based on Symbian DBMS architecture, hence it is a client/server architecture and provides safe access for multiple clients
- Client access to the database is provided by CCommsDatabase

Using Comms Database

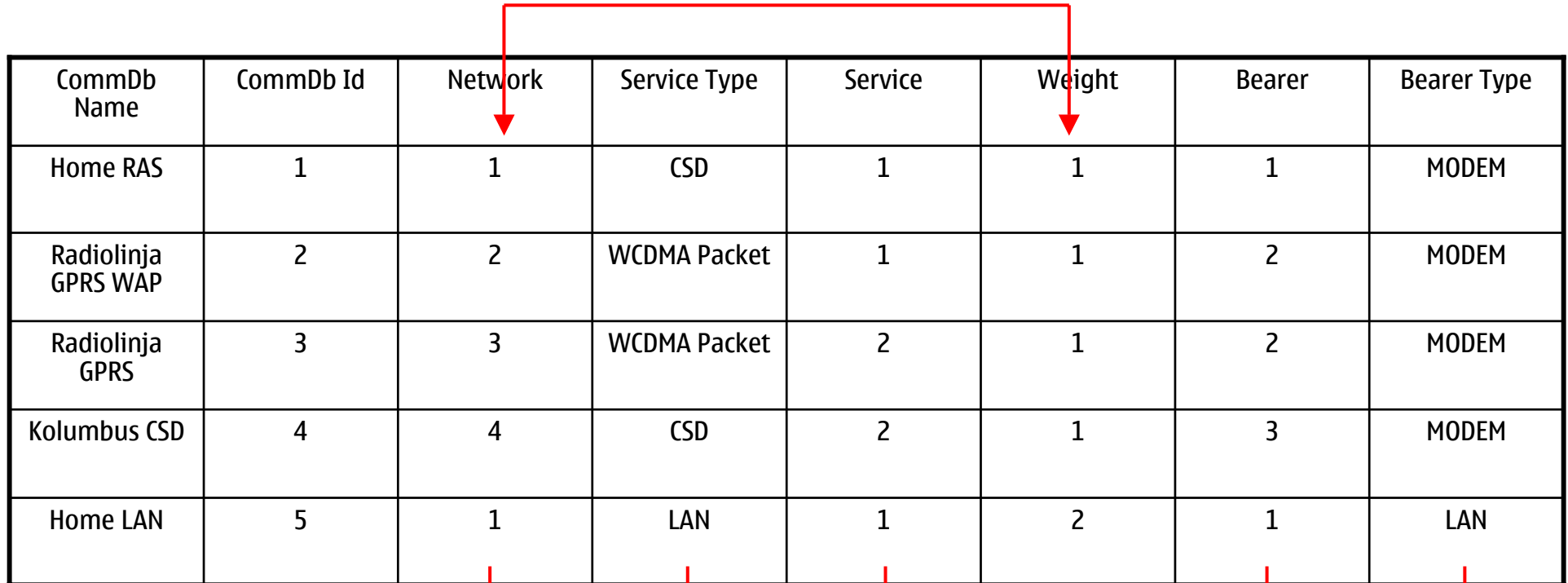
```
CCommsDatabase *db;
CCommsDbTableView *view;
TInt result, count;
TBuf<40> iapName;
TUint32 iapId;

// Open Database
db = CCommsDatabase::NewL();
view = db->OpenTableLC(TPtrC(IAP));

// Walk through records
count = 1;
result = view->GotoFirstRecord();
while (result == KErrNone)
{
    view->ReadTextL(TPtrC(COMMDB_NAME), iapName);
    view->ReadUintL(TPtrC(COMMDB_ID), iapId);
    ui->Print(iapName, iapId);
    result = view->GotoNextRecord();
}

delete view;
delete db;
```

Example IAP Table



CommDb Name	CommDb Id	Network	Service Type	Service	Weight	Bearer	Bearer Type
Home RAS	1	1	CSD	1	1	1	MODEM
Radiolinja GPRS WAP	2	2	WCDMA Packet	1	1	2	MODEM
Radiolinja GPRS	3	3	WCDMA Packet	2	1	2	MODEM
Kolumbus CSD	4	4	CSD	2	1	3	MODEM
Home LAN	5	1	LAN	1	2	1	LAN

References

Network
Table

Service
Type
Tables

Specific
Service Type
Table

Specific
Bearer
Table

Bearer
Tables

Outline

- Introduction to Networking
- Connection Management
- Socket Communication
- HTTP Client API
- Communications Database
- **More Information**

More Information

- Symbian OS Communication Programming
 - Jipping, Michael J
 - ISBN 0-470-84430-2
- www.forum.nokia.com/symbian
 - Series 60 Developer Platform 2.0: IAP Connect Example v2.0
 - Series 60 Developer Platform 2.0: HTTP Client Example v1.0
- SDK: Sockets example



Thank You!